

Examen blanc

CERTIFICATION TOSA • PYTHON

Cet examen blanc reproduit la progression du TOSA Python, dont le score s'établit sur une échelle de 1 à 1000. Les 35 questions sont organisées en quatre paliers de difficulté croissante, du niveau basique au niveau expert. Chaque question comporte une seule bonne réponse.

Durée conseillée	45 minutes
Nombre de questions	35 (QCM, une seule bonne réponse)
Notation	1 point par bonne réponse, pas de point négatif
Matériel	Aucun — sans interpréteur ni documentation
Corrigé	En fin de document (à partir de la page « Corrigé »)

Grille d'auto-évaluation indicative

Bonnes réponses	Niveau TOSA estimé	Score indicatif /1000
0 à 12	Initial / Basique	350 – 550
13 à 20	Opérationnel	551 – 725
21 à 28	Avancé	726 – 875
29 à 35	Expert	876 – 1000

Le score réel du TOSA est calculé par un algorithme adaptatif : cette grille reste donc une estimation pédagogique destinée à situer le candidat.

NIVEAU BASIQUE — questions 1 à 8

Syntaxe fondamentale, types, opérateurs et fonctions intégrées.

Question 1. Quel mot-clé permet de définir une fonction en Python ?

- a) function
- b) def
- c) func
- d) define

Question 2. Quelle est la valeur de l'expression `7 // 2` ?

- a) 3.5
- b) 3
- c) 4
- d) 3.0

Question 3. Quel est le type renvoyé par `type(3.0)` ?

- a) int
- b) float
- c) double
- d) decimal

Question 4. Comment écrit-on un commentaire sur une seule ligne ?

- a) `// commentaire`
- b) `/* commentaire */`
- c) `# commentaire`
- d) `-- commentaire`

Question 5. Que produit `print("Python"[0])` ?

- a) P
- b) Python
- c) 0
- d) Une erreur

Question 6. Quel opérateur correspond à l'exponentiation (puissance) ?

- a) `^`
- b) `**`
- c) `//`
- d) `pow`

Question 7. Quelle instruction convertit la chaîne `"42"` en entier ?

- a) `str(42)`
- b) `int("42")`
- c) `integer("42")`
- d) `"42".toInt()`

Question 8. Quel est le résultat de `len("Bonjour")` ?

- a) 6
- b) 7
- c) 8
- d) Une erreur

NIVEAU OPÉRATIONNEL — questions 9 à 18

Structures de données, slicing, méthodes de listes/dictionnaires et boucles.

Question 9. Soit `L = [10, 20, 30, 40, 50]`. Que renvoie `L[1:3]` ?

- a) `[10, 20, 30]`
- b) `[20, 30]`
- c) `[20, 30, 40]`
- d) `[10, 20]`

Question 10. Soit `L = [1, 2, 3]`. Quelle méthode ajoute l'élément 4 à la fin de la liste ?

- a) `L.add(4)`
- b) `L.append(4)`
- c) `L.insert(4)`
- d) `L.push(4)`

Question 11. Sur `L = [1, 2, 3]`, quelle est la différence entre `L.append([4, 5])` et `L.extend([4, 5])` ?

- a) Aucune différence
- b) `append` insère la liste comme un seul élément ; `extend` ajoute chaque élément séparément
- c) `extend` lève une erreur
- d) `append` fusionne, `extend` imbrique

Question 12. Soit `d = {"a": 1, "b": 2}`. Que renvoie `d.get("c", 0)` ?

- a) `None`
- b) Une erreur `KeyError`
- c) 0
- d) `"c"`

Question 13. Que renvoie `list(range(2, 10, 2))` ?

- a) `[2, 4, 6, 8]`
- b) `[2, 4, 6, 8, 10]`
- c) `[2, 3, 4, ..., 9]`
- d) `[0, 2, 4, 6, 8]`

Question 14. Quelle affirmation est vraie à propos des tuples ?

- a) Ils sont modifiables
- b) Ils sont immuables
- c) Ils n'acceptent qu'un seul type
- d) Ils s'écrivent avec des accolades

Question 15. Que renvoie `len({1, 2, 2, 3, 3, 3})` ?

- a) 6
- b) 3
- c) 2
- d) Une erreur

Question 16. Que renvoie `"a,b,c".split(",")` ?

- a) "abc"
- b) ["a", "b", "c"]
- c) ("a", "b", "c")
- d) ["a,b,c"]

Question 17. Quel est le résultat de l'expression `3 in [1, 2, 3, 4]` ?

- a) True
- b) False
- c) 3
- d) 2

Question 18. Que fait l'instruction `break` dans une boucle ?

- a) Passe à l'itération suivante
- b) Interrompt immédiatement la boucle
- c) Redémarre la boucle
- d) Met la boucle en pause

NIVEAU AVANCÉ — questions 19 à 28

Compréhensions, lambda, exceptions, POO, fichiers et arguments variables.

Question 19. Que produit `[x**2 for x in range(4)]` ?

- a) [0, 1, 4, 9]
- b) [1, 4, 9, 16]
- c) [0, 1, 2, 3]
- d) [2, 4, 6, 8]

Question 20. Que définit l'expression `lambda x: x + 1` ?

- a) Une variable
- b) Une fonction anonyme qui ajoute 1 à son argument
- c) Une erreur de syntaxe
- d) Une boucle

Question 21. Dans une structure `try / except / finally`, quel bloc est toujours exécuté ?

- a) try
- b) except
- c) finally
- d) else

Question 22. Dans une méthode d'instance, à quoi sert le premier paramètre self ?

- a) Il référence la classe
- b) Il référence l'instance courante de l'objet
- c) C'est la valeur de retour
- d) Il est purement décoratif

Question 23. Quelle méthode spéciale sert à initialiser les attributs d'un nouvel objet ?

- a) `__new__`
- b) `__init__`
- c) `__create__`
- d) `__call__`

Question 24. Quel est l'intérêt principal de `with open("f.txt") as f:` ?

- a) Ouvrir le fichier plus rapidement
- b) Fermer automatiquement le fichier en sortie de bloc
- c) Éviter de lire le fichier
- d) Créer le fichier s'il manque

Question 25. Dans `def f(*args, **kwargs):`, que représente `**kwargs` ?

- a) Une liste d'arguments positionnels
- b) Un dictionnaire d'arguments nommés
- c) Un tuple d'arguments
- d) Un ensemble (set)

Question 26. Que renvoie `list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6]))` ?

- a) `[1, 3, 5]`
- b) `[2, 4, 6]`
- c) `[1, 2, 3, 4, 5, 6]`
- d) `[]`

Question 27. Que produit `{x: x**2 for x in range(3)}` ?

- a) `{0: 0, 1: 1, 2: 4}`
- b) `[0, 1, 4]`
- c) `(0, 1, 4)`
- d) `{0, 1, 4}`

Question 28. Quelle est la différence entre `==` et `is` ?

- a) Aucune
- b) `==` compare les valeurs, `is` compare l'identité (même objet en mémoire)
- c) `is` compare les valeurs
- d) `==` est simplement plus rapide

NIVEAU EXPERT — questions 29 à 35

Générateurs, décorateurs, GIL, complexité, gestionnaires de contexte et pièges du langage.

Question 29. Quel mot-clé transforme une fonction en générateur ?

- a) return
- b) yield
- c) async
- d) gen

Question 30. Qu'est-ce qu'un décorateur en Python ?

- a) Un commentaire spécial
- b) Une fonction qui prend une fonction et renvoie une fonction modifiée
- c) Une classe abstraite
- d) Un type de boucle

Question 31. Que produit ce code lorsqu'il est exécuté ?

```
def f(x, L=[]):  
    L.append(x)  
    return L  
print(f(1))  
print(f(2))
```

- a) [1] puis [2]
- b) [1] puis [1, 2]
- c) [1, 2] puis [1, 2]
- d) Une erreur

Question 32. Que désigne le GIL (Global Interpreter Lock) dans CPython ?

- a) Un gestionnaire de mémoire
- b) Un verrou qui empêche plusieurs threads d'exécuter du bytecode Python simultanément
- c) Une bibliothèque graphique
- d) Un système de journalisation

Question 33. Quelle est la complexité temporelle moyenne d'un accès par clé dans un dictionnaire ?

- a) $O(n)$
- b) $O(\log n)$
- c) $O(1)$
- d) $O(n^2)$

Question 34. Quelle paire de méthodes faut-il implémenter pour qu'une classe serve de gestionnaire de contexte (utilisable avec with) ?

- a) `__start__` et `__stop__`
- b) `__enter__` et `__exit__`
- c) `__open__` et `__close__`
- d) `__init__` et `__del__`

Question 35. Que produit ce code ?

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
b[0][0] = 99
print(a[0][0])
```

- a)** 1
- b)** 99
- c)** Une erreur
- d)** None

Corrigé détaillé

RÉPONSES & EXPLICATIONS

Récapitulatif des réponses

1: b	2: b	3: b	4: c	5: a	6: b	7: b
8: b	9: b	10: b	11: b	12: c	13: a	14: b
15: b	16: b	17: a	18: b	19: a	20: b	21: c
22: b	23: b	24: b	25: b	26: b	27: a	28: b
29: b	30: b	31: b	32: b	33: c	34: b	35: b

Question 1

Bonne réponse : b) def

Le mot-clé `def` introduit la définition d'une fonction. `function`, `func` et `define` ne sont pas des mots-clés Python.

Question 2

Bonne réponse : b) 3

L'opérateur `//` réalise la division entière (floor division) et renvoie un entier ici : 3. L'opérateur `/` aurait donné 3.5.

Question 3

Bonne réponse : b) float

Tout littéral numérique comportant un point décimal est de type `float`. Python ne possède pas de type `double` ; `decimal` existe mais via le module `decimal`.

Question 4

Bonne réponse : c) # commentaire

En Python, le caractère `#` introduit un commentaire jusqu'à la fin de la ligne. Les autres syntaxes viennent d'autres langages.

Question 5

Bonne réponse : a) P

Une chaîne est indexable : l'indice 0 correspond au premier caractère, donc 'P'. L'indexation commence toujours à 0.

Question 6

Bonne réponse : b) **

L'opérateur `**` élève à la puissance ($2 ** 3 = 8$). Le caractère `^` est l'opérateur OU exclusif bit à bit, pas une puissance.

Question 7

Bonne réponse : b) int("42")

La fonction `int()` convertit une chaîne représentant un nombre entier en type `int`. `str()` ferait l'inverse.

Question 8

Bonne réponse : b) 7

`len()` renvoie le nombre de caractères : B-o-n-j-o-u-r soit 7 caractères.

Question 9

Bonne réponse : b) [20, 30]

Le slicing `L[1:3]` sélectionne les indices 1 et 2 (la borne de fin 3 est exclue), soit [20, 30].

Question 10

Bonne réponse : b) L.append(4)

append() ajoute un élément en fin de liste. add() concerne les sets, insert() exige un indice, et push() n'existe pas sur les listes.

Question 11

Bonne réponse : b) append insère la liste comme un seul élément ; extend ajoute chaque élément séparément

append([4,5]) donne [1,2,3,[4,5]] (un élément imbriqué) ; extend([4,5]) donne [1,2,3,4,5] (concaténation élément par élément).

Question 12

Bonne réponse : c) 0

get() renvoie la valeur par défaut fournie (ici 0) quand la clé est absente, au lieu de lever KeyError comme le ferait d["c"].

Question 13

Bonne réponse : a) [2, 4, 6, 8]

range(2, 10, 2) part de 2, avance par pas de 2, et s'arrête avant 10 : 2, 4, 6, 8. La borne de fin est exclue.

Question 14

Bonne réponse : b) Ils sont immuables

Un tuple est immuable : une fois créé, on ne peut ni ajouter, ni retirer, ni modifier ses éléments. Les accolades désignent dict et set.

Question 15

Bonne réponse : b) 3

Un set ne conserve que des valeurs uniques : {1, 2, 3} contient 3 éléments. Les doublons sont automatiquement éliminés.

Question 16

Bonne réponse : b) ["a", "b", "c"]

split() découpe une chaîne selon le séparateur fourni et renvoie une liste de sous-chaînes.

Question 17

Bonne réponse : a) True

L'opérateur in teste l'appartenance et renvoie un booléen. 3 est bien présent dans la liste, donc True.

Question 18

Bonne réponse : b) Interrompt immédiatement la boucle

break sort définitivement de la boucle en cours. C'est continue qui passe à l'itération suivante.

Question 19

Bonne réponse : a) [0, 1, 4, 9]

range(4) parcourt 0, 1, 2, 3 ; la compréhension élève chacun au carré : 0, 1, 4, 9.

Question 20

Bonne réponse : b) Une fonction anonyme qui ajoute 1 à son argument

lambda crée une fonction anonyme (sans nom) en une ligne. Ici elle prend x et renvoie x + 1.

Question 21

Bonne réponse : c) finally

Le bloc finally s'exécute systématiquement, qu'une exception ait été levée ou non. Il sert typiquement à libérer des ressources.

Question 22

Bonne réponse : b) Il référence l'instance courante de l'objet

self est une référence à l'instance sur laquelle la méthode est appelée ; il permet d'accéder aux attributs et méthodes de cet objet.

Question 23

Bonne réponse : b) `__init__`

`__init__` est l'initialiseur (souvent appelé constructeur) : il reçoit `self` et définit les attributs au moment de l'instanciation.

Question 24

Bonne réponse : b) Fermer automatiquement le fichier en sortie de bloc

`with` utilise un gestionnaire de contexte qui ferme automatiquement le fichier, même en cas d'exception. Plus besoin d'appeler `f.close()`.

Question 25

Bonne réponse : b) Un dictionnaire d'arguments nommés

`**kwargs` collecte les arguments passés par mot-clé dans un dictionnaire. `*args` collecte, lui, les arguments positionnels dans un tuple.

Question 26

Bonne réponse : b) [2, 4, 6]

`filter` conserve uniquement les éléments pour lesquels la fonction renvoie `True`. Ici la condition garde les nombres pairs : 2, 4, 6.

Question 27

Bonne réponse : a) {0: 0, 1: 1, 2: 4}

Il s'agit d'une compréhension de dictionnaire : chaque clé `x` est associée à sa valeur `x` au carré.

Question 28

Bonne réponse : b) `==` compare les valeurs, `is` compare l'identité (même objet en mémoire)

`==` teste l'égalité des valeurs ; `is` teste si deux noms désignent le même objet (même adresse mémoire). Deux listes égales ne sont pas forcément le même objet.

Question 29

Bonne réponse : b) `yield`

`yield` suspend l'exécution en produisant une valeur et reprend là où elle s'était arrêtée à l'itération suivante : c'est ce qui définit un générateur.

Question 30

Bonne réponse : b) Une fonction qui prend une fonction et renvoie une fonction modifiée

Un décorateur enveloppe une fonction (ou une classe) pour en modifier ou enrichir le comportement, sans toucher à son code. La syntaxe `@` en est le sucre syntaxique.

Question 31

Bonne réponse : b) [1] puis [1, 2]

Piège classique : la valeur par défaut `L=[]` est créée une seule fois, à la définition de la fonction, et partagée entre tous les appels. Le second appel réutilise donc la même liste.

Question 32

Bonne réponse : b) Un verrou qui empêche plusieurs threads d'exécuter du bytecode Python simultanément

Le GIL est un verrou global qui limite l'exécution du bytecode à un seul thread à la fois dans CPython. Il pénalise le multithreading CPU-bound mais simplifie la gestion mémoire.

Question 33

Bonne réponse : c) $O(1)$

Un dictionnaire repose sur une table de hachage : l'accès, l'insertion et la suppression par clé sont en $O(1)$ en moyenne.

Question 34

Bonne réponse : b) `__enter__` et `__exit__`

`__enter__` est appelée à l'entrée du bloc `with` (et renvoie la ressource), `__exit__` à la sortie (y compris en cas d'exception) pour le nettoyage.

Question 35

Bonne réponse : b) 99

`copy.copy()` réalise une copie superficielle : la liste externe est dupliquée mais les sous-listes restent partagées. Modifier `b[0][0]` affecte donc aussi `a`. `copy.deepcopy()` aurait isolé les deux structures.